

Lab1_DataLab

CSAPP Data Lab 实验报告

一 检查结果

1 运算符检查

```
● whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab1_DataLab_25803050113$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 12/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 29/48 operators
mul5Sat: 20/30 operators
classifyAdd3: 27/52 operators
floatScaleThreeHalves: 44/60 operators
floatRoundEven: 37/65 operators
float_i2f: 33/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

2 功能测试

```
● whalerealnot@localhost:~/ICS_jc26fall/CSAPP_Lab1_DataLab_25803050113$ ./btest
```

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse
Total points: 110/110			

二 各函数实现思路

P1 signMask

把常数 1 左移 31 位，得到只有最高位为 1 的 `0x80000000`。这个值既是符号位掩码，也是 32 位补码中 `INT_MIN` 的位模式。

P2 bitXor

异或在两个输入位不同时为 1。利用德摩根律把 `x | y` 改写为 `~(~x & ~y)`，再排除如果 `x` 和 `y` 同时为 1 的情况。

P3 negativePart

算术右移 `x` 的符号位，可以得到全 0 或全 1 的掩码，负数利用 `~x + 1` 得到相反数，再与符号掩码相与，非负数会被全 0 的掩码清零。

P4 copyByteWithin

先把 `src` 和 `dst` 乘以 8，得到对应的位移量，再把传入的字节右移，用 `0xFF` 取出。目的字节先用取反掩码清零，再把源字节左移，用按位或合并。

P5 logicalShift

有符号右移负数会在左侧补 1，因此先执行算术右移，再构造右移的几位为 0、其余位为 1 的掩码，从而把补入的符号位清掉。如果 `n` 为 0，掩码仍然是全 1。

P6 swapNibblePairs

使用 `0x0F0F0F0F` 来把字节的两个部分拆分，低四位左移 4 位，高四位右移 4 位并再次与掩码相与，最后把两部分按位或合并。

P7 secondLowestZeroBit

先对 `x` 取反，利用 `v & (v - 1)` 删除最低的 1，再利用 `v & -v` 单独保留剩余结果中最低的 1，也就是原数第二低的 0。减一和取负分别用补码加法实现。

P8 oddParity

把 32 位依次按照 16、8、4、2、1 位折叠，异或，最终最低位等于所有原始位的异或结果。偶数个 1 得到 0，奇数个 1 得到 1，再用逻辑非得到题目要求的奇校验位。

P9 rotateRightBits

把 `n` 与 31 相与，从而把移动位数限制在 0 到 31。结果由逻辑右移部分和左侧两部分组成，右侧使用逻辑右移掩码，移出的低位则左移 $(32 - n) \bmod 32$ 位，最后用按位或合并。

P10 roundEvenPow2

设舍入单位为 2^n ，一半为 $2^{(n-1)}$ 。先计算商的最低位，判断商的奇偶，再给 `x` 加上 `half - 1 + quotientOdd`，最后清除低 `n` 位。这样正好一半时，偶数商向下保留，奇数商向上进位。

P11 midpointTowardFirst

使用 $(x \& y) + ((x \oplus y) \gg 1)$ 求不直接计算 $x + y$ 的向下中点。如果和为奇数且 x 大于 y ，应在向下中点上加 1，才能选择更接近第一个参数 x 的整数。比较大小时先处理符号不同的情况，避免只依赖可能溢出的减法符号。

P12 isBetweenEitherOrder

分别判断 $x < a$ 和 $x < b$ 。如果两个判断结果不同，说明两个端点分处 x 两侧。如果 x 等于任意端点，也返回 1。带符号比较同样先判断符号是否不同，符号相同时再检查差值的符号。

P13 mul5Sat

依次计算 $2x$ 、 $4x$ 和 $5x$ ，并检查每次加法前后的符号关系。只要任意一步发生溢出，就根据原始 x 的符号选择 INT_MAX 或 INT_MIN ；否则返回正常的 $5x$ 。

P14 classifyAdd3

第三个数可能把精确结果带回合法范围，所以不能只检查中间一次有符号溢出。把结果拆成 high 和 low 两部分， low 保存最低 32 位， high 累加三个输入的符号扩展和两次无符号进位。 high 与 low 的符号扩展之差正好给出 1、0 或 -1。

P15 floatScaleThreeHalves

先拆出符号、指数和小数字段。NaN 与无穷大原样返回；非规格化数直接对小数字段计算 $3/2$ ；规格化数先补回隐藏位，再计算有效数字的三倍，并根据规格化要求右移 1 或 2 位。被丢弃部分按照最近偶数规则舍入，指数溢出时返回无穷大。

P16 floatRoundEven

根据指数判断小数点位置。绝对值小于 0.5 时，返回带原符号的零；指数足够大时，原数已经是整数。其余情况清除有效数字中的小数位，比较被删除部分与一半的大小。正好一半时，根据保留整数的最低位选择偶数。

P17 float_i2f

先保存符号，并使用无符号数取得绝对值，从高到低寻找最高的 1，以此确定浮点指数。有效位不超过 24 位时可以直接对齐；超过时右移并记录被删除部分，使用最近偶数规则舍入。还要处理舍入后有效数字产生新最高位的情况。

P18 bitCount

使用分组并行计数。先用 $0x55555555$ 统计每两位，再用 $0x33333333$ 合并为每四位的计数，随后依次合并为 8、16 和 32 位计数。因为每一步只合并互不重叠的分组，所以不会重复统计。

P19 bitReverse

按照 1、2、4、8、16 位的粒度逐级交换相邻分组。掩码分别选出每组的低半部分，低半部分左移、高半部分右移，再用按位或合并。五轮交换后，原来的 `bit i` 会移动到 `bit 31-i`。

四 参考资料与工具

1. Randal E. Bryant, David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*, 3rd Edition, Chapter 2.
2. Bilibili 郭郭wg-深入理解计算机系统 实验1 CSAPP】 datalab + 环境搭建 data lab
3. OpenAI Codex：用于概念解释、实现思路讨论以及编译报错排查。